

- SRE Platform Engineer — Interview Questions & Answers (Part 1)
  - Section 1: SRE Fundamentals & Philosophy (Q1–Q7)
    - Q1. What is SRE and how does it differ from traditional DevOps?
    - Q2. Explain SLIs, SLOs, SLAs, and Error Budgets with a real example.
    - Q3. What is "toil" in SRE? How do you measure and reduce it?
    - Q4. Describe your incident response process.
    - Q5. What is a blameless postmortem and why is it important?
    - Q6. How do you decide what to alert on vs. what to log vs. what to ignore?
    - Q7. How do you implement SLO-based alerting?
  - Section 2: Kubernetes & Container Orchestration (Q8–Q15)
    - Q8. Explain the Kubernetes architecture. What happens when you run `kubectl apply`?
    - Q9. How do you debug a pod stuck in `CrashLoopBackOff`?
    - Q10. How do you implement zero-downtime deployments in Kubernetes?
    - Q11. Explain Kubernetes networking: Services, Ingress, and NetworkPolicies.
    - Q12. How do you right-size Kubernetes resource requests and limits?
    - Q13. Explain HPA, VPA, and Karpenter. When do you use each?
    - Q14. How do you manage Kubernetes secrets securely?
    - Q15. How do you implement Pod Security in modern Kubernetes (post-`PodSecurityPolicy`)?

# SRE Platform Engineer — Interview Questions & Answers (Part 1)

**Role:** Site Reliability Engineer / Platform Engineering | **Level:** 7-10 Years | **Type:** L1/L2 Interview

## Section 1: SRE Fundamentals & Philosophy (Q1–Q7)

# Q1. What is SRE and how does it differ from traditional DevOps?

Answer:

SRE (Site Reliability Engineering) is Google's approach to operations — treating ops problems as software engineering problems. DevOps is a culture and set of practices; SRE is a **specific implementation** of DevOps with prescriptive practices.

| Aspect   | DevOps                             | SRE                                                   |
|----------|------------------------------------|-------------------------------------------------------|
| Focus    | Culture, collaboration, automation | Reliability, engineering, measurement                 |
| Metrics  | Deployment frequency, lead time    | SLIs, SLOs, error budgets                             |
| Toil     | Automate what you can              | Measure toil, cap at 50% of time                      |
| On-call  | Varies by team                     | Structured, blameless, with error budget consequences |
| Risk     | "Move fast"                        | "Move fast within error budget"                       |
| Staffing | Ops-minded engineers               | Software engineers doing ops                          |

**Key SRE principle:** "Hope is not a strategy." Every reliability decision is data-driven through SLIs/SLOs, not gut feeling.

# Q2. Explain SLIs, SLOs, SLAs, and Error Budgets with a real example.

Answer:

**SLI (Service Level Indicator):** A quantitative measure of service behavior.

- Example:  $\text{successful requests} / \text{total requests} = \text{availability}$
- Example:  $\text{requests served} < 300\text{ms} / \text{total requests} = \text{latency}$

**SLO (Service Level Objective):** A target value for an SLI.

- Example: "99.9% of requests succeed over a 30-day rolling window"
- Example: "95% of requests complete in < 300ms"

**SLA (Service Level Agreement):** A contract with consequences (usually financial) if the SLO is breached.

- Example: "If availability drops below 99.5%, customer gets 10% credit"
- SLA is always **less strict** than internal SLO (buffer)

**Error Budget:** The inverse of SLO — how much unreliability you're allowed.

- 99.9% SLO = 0.1% error budget = **43 minutes of downtime per month**
- 99.95% SLO = 0.05% error budget = **21.6 minutes per month**
- 99.99% SLO = 0.01% error budget = **4.3 minutes per month**

**How I use error budgets:**

- Budget remaining > 50%: Ship features aggressively, take risks
  - Budget remaining 20-50%: Ship carefully, increase testing
  - Budget remaining < 20%: Freeze feature releases, focus on reliability
  - Budget exhausted: All engineering shifts to reliability work
- 

## Q3. What is "toil" in SRE? How do you measure and reduce it?

**Answer:**

**Toil** is manual, repetitive, automatable, tactical, devoid of lasting value work that scales linearly with service growth.

**Examples of toil:**

- Manually restarting pods after crashes
- Hand-editing config files for each environment
- Running deployment scripts manually
- Manually scaling infrastructure for traffic spikes
- Responding to alerts that could be auto-remediated

**NOT toil:** Incident response (requires human judgment), architecture reviews, automation work.

**Google's rule:** SRE teams should spend **≤50% of time on toil**. If toil exceeds 50%, stop taking on new services and automate.

### How I measure toil:

1. Track time spent on operational tasks weekly (simple spreadsheet or Jira labels)
2. Categorize: automatable vs. requires judgment
3. Calculate toil percentage: **toil hours / total hours**
4. Prioritize automation by: frequency × time-per-occurrence × risk

### Reduction strategies:

- Self-healing systems (auto-restart, auto-scale)
  - Runbook automation (PagerDuty + Lambda for auto-remediation)
  - Self-service platforms (developers deploy without SRE involvement)
  - Eliminate alerts that don't require action (reduce noise)
- 

## Q4. Describe your incident response process.

### Answer:

#### Phase 1: Detection (0-5 min)

- Alert fires (PagerDuty/OpsGenie) → On-call engineer acknowledges
- Check dashboard for scope: how many users affected? which services?
- Classify severity:
  - **SEV1:** Customer-facing outage, revenue impact → all-hands war room
  - **SEV2:** Degraded performance, partial outage → primary on-call + backup
  - **SEV3:** Non-critical issue → on-call handles during business hours

#### Phase 2: Mitigation (5-30 min)

- **Mitigate first, debug later.** Goal is to restore service, not find root cause.
- Common actions: rollback deployment, scale up, failover to DR, toggle feature flag, restart pods
- Communicate: Status page update, Slack channel update every 15 min

## Phase 3: Resolution

- Apply permanent fix once mitigation stabilizes
- Verify with monitoring that SLIs return to normal

## Phase 4: Post-Incident Review (within 48 hours)

- **Blameless postmortem** — focus on systems, not people
- Document: timeline, root cause, impact (error budget consumed), what went well, what didn't
- Action items with owners and deadlines
- Share with entire engineering org (learning culture)

**Key metric:** MTTR (Mean Time to Recovery). I target <30 min for SEV1. MTTR matters more than MTBF (Mean Time Between Failures) because failures are inevitable.

---

# Q5. What is a blameless postmortem and why is it important?

## Answer:

A blameless postmortem focuses on **what happened** and **how to prevent recurrence**, not **who made the mistake**.

## Structure I follow:

### **## Incident: [Title] – [Date]**

#### **### Impact**

- Duration: 45 minutes
- Users affected: ~12,000
- Error budget consumed: 8% of monthly budget

#### **### Timeline**

- 14:00 – Deployment of v2.3.1 to production
- 14:05 – Error rate spike from 0.1% to 15%
- 14:08 – PagerDuty alert fires
- 14:12 – On-call acknowledges, begins investigation
- 14:18 – Root cause identified: DB migration locked table
- 14:22 – Rollback initiated
- 14:30 – Service restored, error rate normal

#### **### Root Cause**

DB migration ran ALTER TABLE on a hot table without pt-online-schema-change, causing lock contention.

#### **### Contributing Factors**

- No pre-prod load test with migration

- Migration review checklist didn't include lock impact

### ### Action Items

- [ ] Add lock-impact check to migration review (Owner: @alice, Due: May 15)
- [ ] Implement pt-online-schema-change for all DDL (Owner: @bob, Due: May 20)
- [ ] Add DB lock duration alert (Owner: @carol, Due: May 10)

### ### What Went Well

- Alert fired within 3 min
- Rollback was fast (8 min)

**Why blameless matters:** If people fear punishment, they hide mistakes. Hidden mistakes repeat. Blameless culture → people report issues early → faster detection → better reliability.

---

## Q6. How do you decide what to alert on vs. what to log vs. what to ignore?

**Answer:**

**Alert (page someone):** Only if it requires **immediate human action** and affects **user-facing SLOs**.

-  Error rate > 1% for 5 minutes (SLO breach imminent)
-  Latency p99 > 2s for 10 minutes
-  Disk usage > 90%
-  A single pod restart (self-heals)
-  CPU spike that resolves in 30 seconds

**Warn (ticket/Slack):** Issues that need attention but not urgently.

- DLQ has messages
- Certificate expiring in 14 days
- Error budget consumed > 50%

**Log (observe only):** Everything else. Debug info, request traces, audit trails.

**Google's alerting rules I follow:**

1. Every alert must be **actionable** — if you can't do anything, don't alert
2. Every alert must be **urgent** — if it can wait until morning, it's not an alert

3. Every alert should have a **runbook link** — the on-call engineer shouldn't have to guess
4. **Alert on symptoms, not causes** — "users seeing errors" not "CPU is high"

**Anti-pattern:** Alert fatigue. If the team gets 100 alerts/day, they stop paying attention.  
Target: <5 actionable alerts per on-call shift.

---

## Q7. How do you implement SLO-based alerting?

**Answer:**

**Multi-window, multi-burn-rate alerting** (Google's recommended approach):

Instead of alerting on instantaneous error rate, alert based on **error budget burn rate**:

| Window  | Burn Rate | Meaning                     | Action                  |
|---------|-----------|-----------------------------|-------------------------|
| 1 hour  | 14.4x     | Budget consumed in 2 hours  | Page immediately (SEV1) |
| 6 hours | 6x        | Budget consumed in 5 days   | Page (SEV2)             |
| 3 days  | 1x        | Budget consumed on schedule | Ticket (investigate)    |

**Example for 99.9% SLO (43 min/month budget):**

```
# Prometheus alert rule
- alert: HighErrorBudgetBurn
  expr: |
    (
      sum(rate(http_requests_total{status=~"5.."}[1h]))
      / sum(rate(http_requests_total[1h]))
    ) > (14.4 * 0.001) # 14.4x burn rate of 0.1% error budget
  for: 5m
  labels:
    severity: critical
  annotations:
    summary: "Error budget burning 14.4x faster than allowed"
    runbook: "https://wiki/runbooks/high-error-rate"
```

**Why this is better than threshold alerts:** A brief spike (30 seconds of errors) won't page. But sustained elevated errors that threaten the monthly budget will.

---

## Section 2: Kubernetes & Container Orchestration (Q8–Q15)

---

### Q8. Explain the Kubernetes architecture. What happens when you run `kubectl apply`?

Answer:

**Control Plane:**

- **API Server** — Frontend for all K8s operations. All `kubectl` commands hit this.
- **etcd** — Distributed key-value store. Stores all cluster state.
- **Scheduler** — Assigns pods to nodes based on resources, affinity, taints.
- **Controller Manager** — Runs control loops (Deployment controller, ReplicaSet controller, etc.)

**Worker Nodes:**

- **kubelet** — Agent on each node. Ensures pods are running as specified.
- **kube-proxy** — Network proxy. Implements Service abstraction (iptables/IPVS rules).
- **Container Runtime** — `containerd` (or CRI-O). Actually runs containers.

What happens on `kubectl apply -f deployment.yaml`:

1. `kubectl` sends YAML to **API Server** (authenticated via `kubeconfig`)
  2. API Server validates, stores desired state in **etcd**
  3. **Deployment Controller** detects new Deployment, creates ReplicaSet
  4. **ReplicaSet Controller** detects desired replicas, creates Pod objects
  5. **Scheduler** assigns Pods to Nodes (based on resources, affinity, taints)
  6. **kubelet** on assigned Node pulls image, starts container via `containerd`
  7. **kube-proxy** updates iptables for Service routing
  8. **Readiness probe** passes → Pod added to Service endpoints → receives traffic
- 

### Q9. How do you debug a pod stuck in CrashLoopBackOff?



**Answer:**

### Systematic approach:

```
# 1. Get pod status and events
kubectl describe pod <name> -n <ns>
# Look at: Events, Last State, Exit Code, Reason

# 2. Check logs from crashed container
kubectl logs <pod> -n <ns> --previous

# 3. Common exit codes:
# 137 = OOMKilled (need more memory)
# 1   = Application error (check logs)
# 143 = SIGTERM (graceful shutdown failed)
# 0   = Completed (shouldn't restart – check restartPolicy)

# 4. If container won't stay up long enough for logs:
kubectl run debug --image=<same-image> --command -- sleep 3600
kubectl exec -it debug -- /bin/sh
# Manually run the entrypoint to see errors

# 5. Check resource limits
kubectl top pod <name>
# If memory usage near limit → OOMKill incoming

# 6. Check ConfigMaps/Secrets exist
kubectl get configmap,secret -n <ns>

# 7. Check image exists and is pullable
kubectl get events -n <ns> --field-selector reason=Failed
```

### Most common causes in my experience:

1. OOMKilled (memory limit too low) — increase `resources.limits.memory`
2. Missing env var or config — check ConfigMap/Secret references
3. DB connection failure — check NetworkPolicy, endpoint, credentials
4. Readiness probe too aggressive — increase `initialDelaySeconds`
5. Image pull failure — check image tag, registry auth, ECR token expiry

---

## Q10. How do you implement zero-downtime deployments in Kubernetes?

**Answer:**

## Prerequisites (all must be in place):

### 1. Rolling update strategy:

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1          # 1 extra pod during rollout
    maxUnavailable: 0    # Never reduce below desired count
```

2. **Readiness probes:** New pod only receives traffic after health check passes.

### 3. Graceful shutdown:

```
terminationGracePeriodSeconds: 30
lifecycle:
  preStop:
    exec:
      command: ["/bin/sh", "-c", "sleep 5"]
# Sleep 5s allows kube-proxy to update iptables before container stops
```

### 4. PodDisruptionBudget:

```
apiVersion: policy/v1
kind: PodDisruptionBudget
spec:
  minAvailable: 1
  selector:
    matchLabels:
      app: my-app
```

5. **Multiple replicas** spread across AZs (topologySpreadConstraints).

**Common mistake:** No **preStop** hook. Without it, kube-proxy may still route traffic to a terminating pod for a few seconds → 502 errors during deployment.

---

## Q11. Explain Kubernetes networking: Services, Ingress, and NetworkPolicies.

Answer:

## Service types:

| Type         | Access                    | Use Case                   |
|--------------|---------------------------|----------------------------|
| ClusterIP    | Internal only             | Service-to-service         |
| NodePort     | External via node IP:port | Testing/debugging          |
| LoadBalancer | External via cloud LB     | Production external access |
| ExternalName | DNS CNAME                 | Alias to external service  |

## Ingress:

- Layer 7 (HTTP/HTTPS) routing to multiple Services
- Host-based: `api.company.com` → `api-svc`, `web.company.com` → `web-svc`
- Path-based: `/api` → `api-svc`, `/app` → `web-svc`
- TLS termination with certificate
- Requires an Ingress Controller (NGINX, ALB, Traefik)

## NetworkPolicies:

- Default: all pods can talk to all pods (flat network)
- Best practice: **default deny all**, then whitelist specific flows

```
# Deny all ingress+egress
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
```

Then allow specific paths: `app` → database on port 5432, ALB → `app` on port 8080.

---

## Q12. How do you right-size Kubernetes resource requests and limits?

### Answer:

**Step 1: Observe actual usage** (not guess)

```
kubectl top pods -n production
# Or query Prometheus:
# container_memory_usage_bytes{pod="my-app-.*"}
# container_cpu_usage_seconds_total{pod="my-app-.*"}
```

## Step 2: Set requests and limits

- **Request** = guaranteed minimum. Scheduler uses this for placement.
- **Limit** = maximum allowed. Container is OOMKilled (memory) or throttled (CPU) if exceeded.

### My rules:

| Resource | Request                | Limit                                                            |
|----------|------------------------|------------------------------------------------------------------|
| CPU      | p50 usage              | No limit (or 2-4x request). CPU throttling hurts latency.        |
| Memory   | p90 usage + 20% buffer | p99 usage + 30% buffer. OOMKill is worse than over-provisioning. |

### QoS classes:

- **Guaranteed** (request == limit): highest priority, last to be evicted
- **Burstable** (request < limit): moderate priority
- **BestEffort** (no request/limit): first to be evicted — never use in production

**Tools:** Kubernetes VPA (Vertical Pod Autoscaler) in recommend mode — suggests optimal requests/limits based on historical usage.

## Q13. Explain HPA, VPA, and Karpenter. When do you use each?

### Answer:

| Autoscaler | What it scales            | When to use                       |
|------------|---------------------------|-----------------------------------|
| HPA        | Pod replicas (horizontal) | Stateless apps that can scale out |

| Autoscaler | What it scales           | When to use                                                 |
|------------|--------------------------|-------------------------------------------------------------|
| VPA        | Pod resources (vertical) | Stateful apps, batch jobs                                   |
| Karpenter  | Nodes (right-sized)      | Replace Cluster Autoscaler for faster, smarter node scaling |

## HPA:

```
metrics:  
- type: Resource  
  resource:  
    name: cpu  
    target:  
      type: Utilization  
      averageUtilization: 70
```

When average CPU > 70%, add pods. Can also scale on custom metrics (queue depth, request rate).

**VPA:** Adjusts requests/limits per pod. Don't use `updateMode: Auto` with HPA on the same metric (conflict). Use `updateMode: Off` for recommendations only.

## Karpenter vs Cluster Autoscaler:

- CA scales node groups (fixed instance types). Karpenter provisions individual right-sized nodes.
- CA: 2-5 min scale-up. Karpenter: 30-60 seconds.
- Karpenter auto-selects cheapest instance that fits pending pods (Spot-aware).

---

## Q14. How do you manage Kubernetes secrets securely?

### Answer:

**Problem:** K8s Secrets are base64-encoded (not encrypted) in etcd by default.

### Solutions (layered):

1. **Encrypt etcd at rest** — Enable KMS encryption provider in API server config. EKS does this by default with AWS KMS.
2. **External Secrets Operator (ESO):**

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secrets-manager
  target:
    name: db-credentials
  data:
    - secretKey: password
      remoteRef:
        key: prod/database/password
```

Syncs secrets from AWS Secrets Manager / Vault → K8s Secret automatically.

3. **Sealed Secrets (GitOps-safe):** Encrypt secrets client-side. Only the cluster can decrypt. Safe to commit to Git.
4. **IRSA / Workload Identity:** Pods get AWS/GCP credentials without storing keys.
5. **RBAC:** Restrict who can `kubectl get secret` — most developers shouldn't need it.

**My preference:** External Secrets Operator + AWS Secrets Manager. Single source of truth for secrets, auto-rotation, audit trail via CloudTrail.

---

## Q15. How do you implement Pod Security in modern Kubernetes (post-PodSecurityPolicy)?

**Answer:**

PodSecurityPolicy was deprecated in 1.21, removed in 1.25. Replaced by **Pod Security Admission (PSA):**

```
apiVersion: v1
kind: Namespace
metadata:
  name: production
  labels:
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/audit: restricted
    pod-security.kubernetes.io/warn: restricted
```

### Three levels:

| Level             | What it enforces                                                    |
|-------------------|---------------------------------------------------------------------|
| <b>Privileged</b> | No restrictions (system namespaces)                                 |
| <b>Baseline</b>   | Prevents known privilege escalation (no hostNetwork, no privileged) |
| <b>Restricted</b> | Hardened (non-root, drop all capabilities, read-only rootfs)        |

### For production workloads I always enforce:

- `runAsNonRoot: true`
- `allowPrivilegeEscalation: false`
- `readOnlyRootFilesystem: true`
- `capabilities: { drop: ["ALL"] }`
- `seccompProfile: { type: RuntimeDefault }`

**For more granular control:** Use **Kyverno** or **OPA Gatekeeper** as admission controllers. Example: "All images must come from our private ECR registry."